Save Point

Running Locally

QuillFin · Prepared for Logan Young, Save Point Limited · 6 November 2026

This is a local-only app. No accounts, no server, no authentication beyond a single hardcoded local session. See `future_work.md` for what's deliberately deferred.

Setup

Requires the Flutter SDK (this project was built and tested against 3.47.2) and either a connected device, a running emulator, or a way to sideload a built APK.

```
flutter pub get
```

Run the app

```
flutter run
```

Or, for a standalone install without an attached debugger:

```
flutter build apk --debug
# then install build/app/outputs/flutter-apk/app-debug.apk onto a device
```

First run: signing in

A plain `flutter run` builds without cloud config, so the sign-in screen offers only **Continue on this device**: everything stays local.

To try QuillFin accounts and sync, copy `config/example.json` to `config/dev.json` (git-ignored), fill in the Clerk and Convex values from `setup_guide.md`, and run:

```
flutter run --dart-define-from-file=config/dev.json
```

The sign-in screen then also offers **Sign in or create an account**, which opens Clerk in your browser. See `accounts_and_sync.md`.

Configuring an AI provider

Question generation needs an AI API key. Go to the Account tab:

1. Pick a provider: `openai` (covers any OpenAI-compatible Chat Completions endpoint: OpenAI itself, or override the base URL to point at OpenRouter, Groq, a local Ollama instance at `http://localhost:11434/v1`, LM Studio, etc.) or `anthropic`.
2. Enter a model name (e.g. `gpt-4o-mini`, `claude-sonnet-5`) and, if using a non-default endpoint, the base URL override.
3. Enter your API key. It's stored via `flutter_secure_storage` (the platform keychain/keystore), never in plain text.

The app itself is never billed for AI usage; you're calling your own provider account directly with your own key.

Where data lives

- `quillfin.db`: the SQLite database (folders, notes, questions, review history), in the standard app-data folder via `path_provider` (`getApplicationDocumentsDirectory()`), so the exact path is OS-dependent. On web it lives in the browser's IndexedDB instead (see `hosting.md`).
- Non-secret settings (provider, model, base URL, default question format) go in `shared_preferences`, the platform's key-value store.
- The API key lives in the platform's secure storage (Android Keystore, iOS Keychain; encrypted browser storage on web), not as a file.

Known Android build gotchas

Worth knowing before adding or upgrading any plugin with native Android code:

- `android/build.gradle.kts` **forces every plugin subproject to compile against SDK 36**, regardless of what each plugin's own build file hardcodes. This exists because `file_picker` 8.x hardcodes an older `compileSdk` directly in its own build file rather than deriving it from Flutter's default, which otherwise produces a "plugin X requires compiling against version Y" build failure the moment any *other* plugin needs 36+. This is a standard, documented Flutter community workaround, not a hack specific to this project. If a future plugin upgrade needs SDK 37+, bump the hardcoded `36` there (and confirm platform 37 is actually installed via the Android SDK manager before assuming it'll just work).
- **Before bumping any plugin's pubspec version to fix a build error, check whether the *actual* fix is the compileSdk override above instead.** Bumping a plugin version to satisfy one error can cascade into unrelated transitive dependency conflicts (this project hit exactly that: bumping `file_picker` pulled in a `win32` version conflict with `share_plus`, fixing that pulled `flutter_secure_storage` into a *different* SDK-version demand, and one of the versions reached that way, `share_plus` 13.3.0, turned out to have a genuinely broken Android build, referencing Kotlin classes that don't exist in its own published package). The versions currently pinned in `pubspec.yaml` are the ones confirmed to actually build; don't bump them reactively without testing the compileSdk-override angle first.

- Verify any Android-affecting change with `flutter build apk --debug` (fast to run, doesn't need an emulator) before assuming a device-level `flutter run` will succeed.

Run tests

```
flutter analyze
flutter test
```

See `testing.md` for what's covered and why widget tests, not device-input simulation, are the reliable way to verify UI behavior in this project.